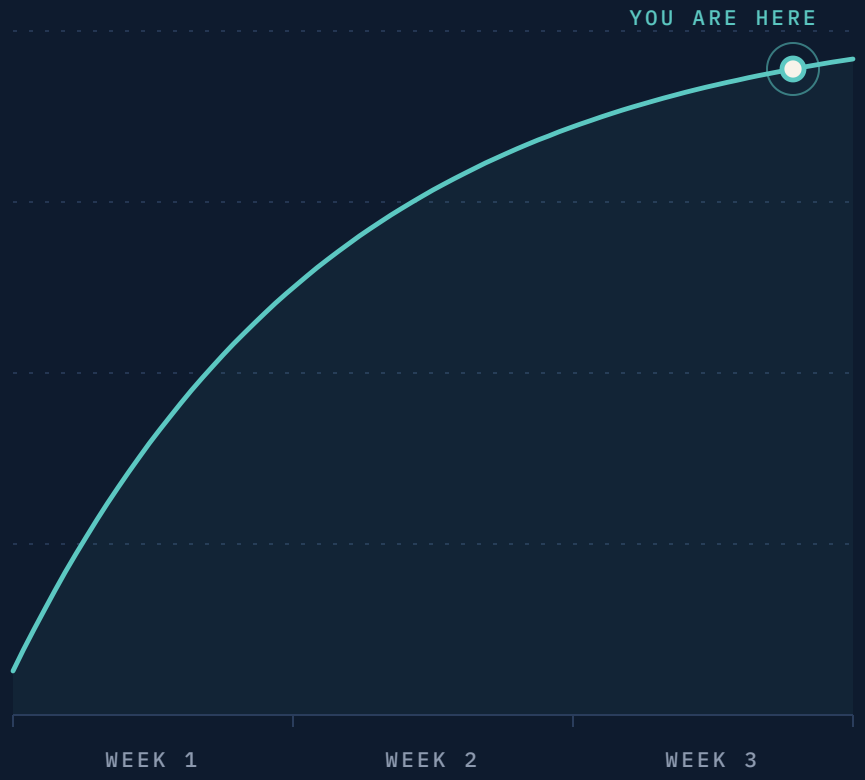
WEEK 3 · DAY 4 · OVERVIEW

Portfolio Aggregation and Scenarios



Created by **Jeff Lenamon**

ABOUT 50 MIN WITH EXERCISES / 8 SECTIONS / 5 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The Desk Lead's Note

The September book, 200 positions. I want the risk report's top lines from your own tested functions: value, duration, spread, DTS, convexity, DV01. Then the what-ifs, repriced in full: every yield -100, -25, +25 and +100 bp; every spread +25 bp and +10 percent. How far off is the duration-and-convexity shortcut? Which sectors carry each loss? Send me the grid as a workbook.

ABOUT 50 MIN WITH EXERCISES / 8 SECTIONS / 5 EXERCISES

Your Route Through Day 4

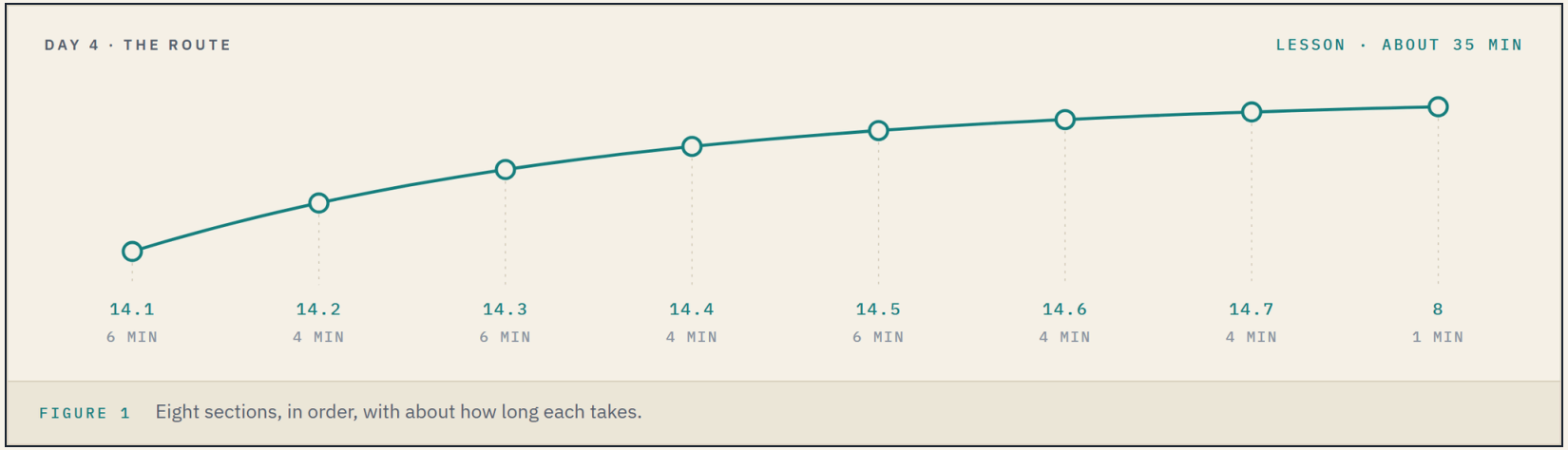


FIGURE 1 Eight sections, in order, with about how long each takes.

	SECTION	ABOUT
14.1	**`weighted_average`**, Docstring First	6 min
14.2	The Book's Risk Card	4 min
14.3	Every Yield Moves, Repriced in Full	6 min
14.4	Every Spread Moves	4 min

	SECTION	ABOUT
14.5	**`price_change_estimate`**, Against the Full Reprice	6 min
14.6	Where Each Loss Lands, by Sector	4 min
14.7	The Grid to the Desk, the Code to Git	4 min
8	Bring It Home	1 min

Time. About 35 minutes for the lesson, then about 15 for the five exercises.

14.1 weighted_average, Docstring First

THE BOOK'S DURATION, AGAINST EXCEL

```
importlib.reload(bondmath)
book_duration_file = bondmath.weighted_average(list(holdings['duration']),
                                               list(holdings['market_value']))
excel_duration = 6.254777411342787
print(f'weighted_average {book_duration_file:.15f}, Excel {excel_duration:.15f}')
```

```
weighted_average 6.254777411342787, Excel 6.254777411342787
```

EXCEL SIDE BY SIDE

JOB	PYTHON	EXCEL	RESULT
Book value	`holdings['market_value'].sum()`	`=SUMPRODUCT(R2:R201,I2:I201+K2:K201)/100`	490,819,215.00
Book duration	`bondmath.weighted_average(...)`	`=SUMPRODUCT(N2:N201,S2:S201)/SUM(S2:S201)`	6.254777411342787

`weighted_average` **returns Excel's SUMPRODUCT over SUM to the last digit: the book's duration is 6.254777 years.**

14.2 The Book's Risk Card

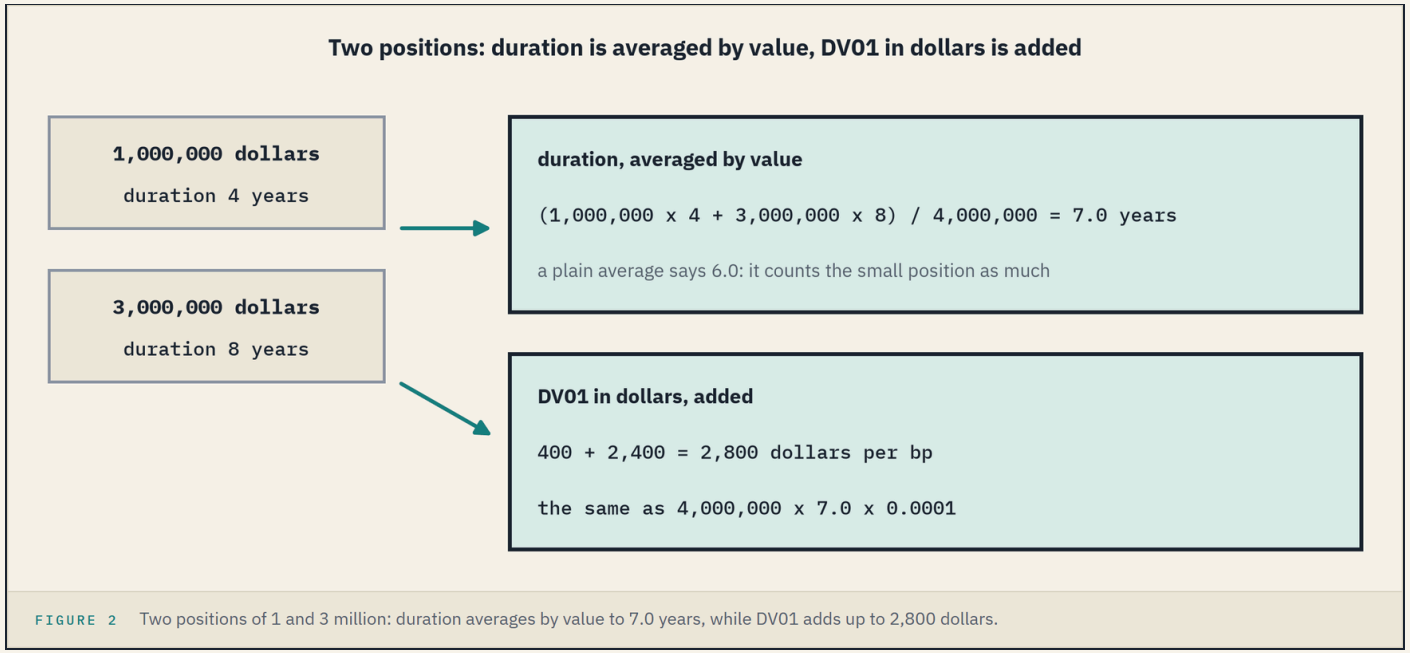


FIGURE 2 Two positions of 1 and 3 million: duration averages by value to 7.0 years, while DV01 adds up to 2,800 dollars.

THE BOOK IN SIX NUMBERS

market value	490,819,215.00 dollars
duration	6.2548 years
spread	180.46 bp
DTS	1064.11 years x bp
convexity	66.9912 years squared
DV01	306,996.11 dollars per bp

Four of the six are averages weighted by value and DV01 is a sum: this card is the base every what-if moves from.

14.3 Every Yield Moves, Repriced in Full

BEZANE, 100 BP HIGHER

```
settlement, maturity, coupon_pct, yield_pct = bonds[0]
before = bondmath.dirty_price(settlement, maturity, coupon_pct, yield_pct)
after = bondmath.dirty_price(settlement, maturity, coupon_pct, yield_pct + 100 / 100)
change_dollars = holdings.loc[0, 'par_held'] * (after - before) / 100
print(f'dirty {before:.4f} at {yield_pct} percent, {after:.4f} at {yield_pct + 1:.3f}')
print(f'the position changes by {change_dollars:,.2f} dollars')
```

```
dirty 108.7099 at 5.606 percent, 103.8202 at 6.606
the position changes by -244,484.16 dollars
```

THE BASE THE CHANGES ARE MEASURED FROM

```
base_value = (holdings['par_held'] * holdings['dirty_bm'] / 100).sum()
print(f'book from the yields  {base_value:,.2f} dollars')
print(f"book from the file    {total_value:,.2f} dollars, {total_value - base_value:,.2f} more")
```

```
book from the yields  490,819,038.14 dollars
book from the file    490,819,215.00 dollars, 176.86 more
```

14.4 Every Spread Moves

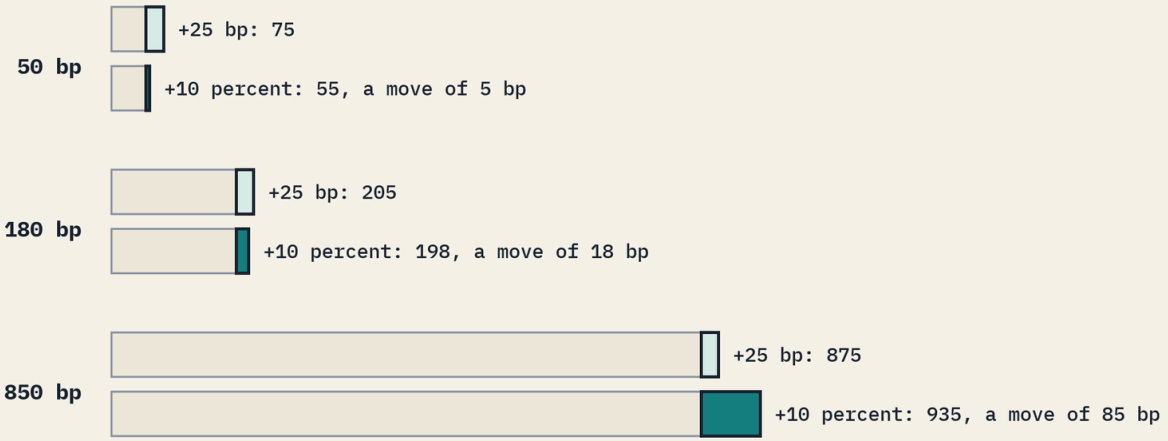


FIGURE 4 Spreads of 50, 180 and 850 bp: +25 bp moves each by 25, +10 percent by 5, 18 and 85.

BOOK DTS GIVES THE 10 PERCENT MOVE

```
from book DTS   -1.0641 percent
full reprice    -1.0493 percent
```

The book's spread is 180.46 bp, so its average move is about 18 bp: less than 25, and so a smaller loss.

14.5 price_change_estimate, Against the Full Reprice

price_change_estimate: Bezane Finance, 100 bp higher, per 100 of face

```
dirty_price x (-modified_duration x dy + 0.5 x convexity x dy ** 2)
```

dirty_price	modified_duration	convexity	dy
108.7099	4.626971	26.3580	100 / 10000 = 0.01
per 100 of face	years	years squared	shift_bp as a decimal



FIGURE 5 Bezane at 100 bp: the duration term of -5.0300 and convexity term of +0.1433 give -4.8867.

BEZANE, 100 BP

full reprice	-4.8897 per 100
duration and convexity	-4.8867 per 100
duration alone	-5.0300 per 100

Duration alone misses by 0.14 per 100; the convexity term takes the miss to 0.003.

The Shortcut Against the Book

THE ESTIMATE, IN THE MODULE

```
def price_change_estimate(dirty_price: float, modified_duration: float, convexity: float, shift_bp: float) -> float:
    dy = shift_bp / 10000
    return dirty_price * (-modified_duration * dy + 0.5 * convexity * dy ** 2)
```

THE BOOK AT 25 BP

```
for shift_bp in [-25, 25]:
    full = totals[f'rates {shift_bp:+d} bp']
    miss = full - estimate([shift_bp] * len(holdings)).sum()
    print(f'{shift_bp:+d} bp: full {full:,.2f}, the shortcut misses by {miss:,.2f} dollars')
```

```
-25 bp: full 7,778,969.05, the shortcut misses by 1,314.62 dollars
+25 bp: full -7,573,434.14, the shortcut misses by -1,283.05 dollars
```

14.6 Where Each Loss Lands, by Sector

DIVIDED BACK

SECTOR	WEIGHT_PCT	DURATION	DTS	DV01_SHARE_PCT
Materials	14.61	5.26	863.68	12.29
Financials	13.90	6.22	1006.25	13.82
Consumer	11.54	6.97	1395.78	12.86
Telecom	11.38	6.16	1090.25	11.20
Industrials	10.70	6.70	1567.20	11.46
Transportation	9.95	8.19	914.17	13.02
Health Care	7.93	5.13	637.25	6.50
Energy	7.91	6.06	1025.17	7.67
Technology	6.18	5.47	869.87	5.40
Utilities	5.91	6.12	1166.57	5.78

EXCEL SIDE BY SIDE

JOB	EXCEL	RESULT
Transportation's duration	the calculated field's cell	8.186803162389122
The whole book	Grand Total	6.254777411342787

14.7 The Grid to the Desk, the Code to Git

THE WORKBOOK

```
with pd.ExcelWriter('scenario_grid.xlsx') as writer:
    grid.round(2).to_excel(writer, sheet_name='scenarios')
    sector_pnl.round(2).to_excel(writer, sheet_name='by_sector')
```

READ IT BACK

```
back = pd.read_excel('scenario_grid.xlsx', sheet_name=None, index_col=0)
assert np.isclose(back['scenarios'], grid.round(2)).all()
assert np.isclose(back['by_sector'], sector_pnl.round(2)).all()
print(list(back.keys()), 'read back, every number as written')
```

```
['scenarios', 'by_sector'] read back, every number as written
```

WHAT GIT SEES

```
 M bondmath.py
  M test_bondmath.py
?? scenario_grid.xlsx
```

STAGED BY HABIT

```
M  bondmath.py
A  scenario_grid.xlsx
M  test_bondmath.py
```

The desk gets the workbook; the record gets the 59 lines that make it.

Bring It Home

COPY AND TEST

```
Set-Location "$HOME\projects\my_bondmath"
$work = "$env:TEMP\pfi_course2_14_k4w7q2xr"
Copy-Item "$work\bondmath.py", "$work\test_bondmath.py", "$work\.gitignore" .
python -m pytest -q
```

CHECK AND COMMIT

```
git diff --stat
git add bondmath.py test_bondmath.py
git status --short
```

If a file slipped in, `git restore --staged <file>` takes it out before the commit.

You Can Now

- Weight a book by market value with **`weighted_average`**, as Excel's **SUMPRODUCT** over **SUM** does.
- Build a book's risk card from your own functions, and say which numbers are averages and which add up.
- Reprice a whole book in full under parallel shifts and under two shapes of spread move, and say what a scenario is and is not.
- Estimate the same moves with **`price_change_estimate`**, and say when the shortcut stops being enough.
- Split each scenario's loss by sector, with **`groupby`** or a PivotTable, and tie the parts back to the book.
- Send a scenario grid as a workbook, and take a file out of the staging area with **`git restore --staged`**.

NEXT

WEEK 3 · DAY 5

Course 2 Capstone

November's extract proved with your own bondmath, cleaned by one function, reconciled, reported, rerun on September, and written up. You write the code.